



Specifying and Analyzing Transactional Consistency Models with Predicates

Hengfeng Wei Si Liu Yuxing Chen

Hunan University

Texas A&M University

Renmin University of China

September 4, 2026

A semantic and graph-theoretic foundation for predicate operations

Item Operations vs. Predicate Operations

Read(x, v)

Read(2, 60)

Write(x, v)

Write(3, 40)

point access
one value

Employee

EID	Salary
0	80
1	100
2	60
3	40

PredRead(P, M)

$P : \text{Salary} < 80$

condition-based access
a set of values

One query can observe both matching and non-matching employees.

What we know about transactional consistency models
without predicates

Review: Intuitive or Operational Specs

Eswaran et al., 1976

The Notions of Consistency and Predicate Locks in a Database System

K.P. Eswaran, J.N. Gray,
R.A. Lorie, and I.L. Traiger
IBM Research Laboratory
San Jose, California

SER

serial execution

Berenson et al., 1995

A Critique of ANSI SQL Isolation Levels

Hal Berenson
Phil Bernstein
Jim Gray
Jim Melton
Elizabeth O'Neil
Patrick O'Neil

Microsoft Corp.
Microsoft Corp.
U.C. Berkeley
Sybase Corp.
UMass/Boston
UMass/Boston

haroldb@microsoft.com
philbe@microsoft.com
gray@crl.com
jim.melton@sybase.com
conell@cs.umb.edu
poneil@cs.umb.edu

4.2 Snapshot Isolation

A transaction executing with *Snapshot Isolation* always reads data from a snapshot of the (committed) data as of the time the transaction started, called its *Start-Timestamp*. This time may be any time before the transaction's first Read. A transaction running in Snapshot Isolation is never blocked attempting a read as long as the snapshot data from its *Start-Timestamp* can be maintained. The transaction's writes (updates, inserts, and deletes) will also be reflected in this snapshot, to be read again if the transaction accesses (i.e., reads or updates) the data a second time. Updates by other transactions active after the transaction *Start-Timestamp* are invisible to the transaction.

Snapshot Isolation is a type of multiversion concurrency control. It extends the Multiversion Mixed Method described in [BHG], which allowed snapshot reads by read-only transactions.

When the transaction T1 is ready to commit, it gets a *Commit-Timestamp*, which is larger than any existing *Start-Timestamp* or *Commit-Timestamp*. The transaction successfully commits only if no other transaction T2 with a *Commit-Timestamp* in T1's interval [*Start-Timestamp*, *Commit-Timestamp*] wrote data that T1 also wrote. Otherwise, T1 will abort. This feature, called *First-commit-ter-wins* prevents lost updates (phenomenon P4). When T1 commits, its changes become visible to all transactions whose *Start-Timestamps* are larger than T1's *Commit-Timestamp*.

SI snapshot + write-conflict control

Review: Axiomatic Specs

A shared execution vocabulary: (VIS, AR)

A Framework for Transactional Consistency Models with Atomic Visibility

Andrea Cerone, Giovanni Bernardi, and Alexey Gotsman

IMDEA Software Institute, Madrid, Spain

atomic visibility

Seeing is Believing: A Client-Centric Specification of Database Isolation

Natasha Crooks
The University of Texas at Austin and Cornell University
Lorenzo Alvisi
The University of Texas at Austin and Cornell University

Yasser Pu
Cornell University
Allen Clement
Google, Inc.

client-centric views

On the Complexity of Checking Transactional Consistency

RANADEEP BISWAS, Université de Paris, IRIF, CNRS, France
CONSTANTIN ENEA, Université de Paris, IRIF, CNRS, France

checking complexity

Visibility determines observable item versions; arbitration orders writes.

Cerone, Bernardi, and Gotsman, 2015; Crooks et al., 2017

Review: Extract Dependency Graphs from Histories

The Notions of Consistency and Predicate Locks in a Database System

K.P. Eswaran, J.N. Gray,
R.A. Lorie, and I.L. Traiger
IBM Research Laboratory
San Jose, California

CONCURRENCY CONTROL FOR DATABASE SYSTEMS
R.E. Stearns, P.M. Lewis II and D.J. Rosenkrantz
General Electric Research & Development Center
Schenectady, N. Y. 12345

EXTENDED ABSTRACT

History

⇒ item dependency graph ⇒

WR, WW, RW edges encode observed
versions and overwrites.

Eswaran et al., 1976; Bernstein, Hadzilacos, and Goodman, 1987

Review: SER Characterization Theorem

The Notions of Consistency and Predicate Locks in a Database System

K.P. Eswaran, J.N. Gray,
R.A. Lorie, and I.L. Traiger
IBM Research Laboratory
San Jose, California

CONCURRENCY CONTROL FOR DATABASE SYSTEMS

R.E. Stearns, P.M. Lewis II and D.J. Rosenkrantz

General Electric Research & Development Center
Schenectady, N. Y. 12345

EXTENDED ABSTRACT

Theorem

An item-operation history is serializable iff its dependency graph is internally consistent and acyclic.

Bernstein, Hadzilacos, and Goodman, 1987; Adya, 1999

Review: SI Characterization Theorem

4.2 Snapshot Isolation

A transaction executing with *Snapshot Isolation* always reads data from a snapshot of the (committed) data as of the time the transaction started, called its *Start-Timestamp*. This time may be any time before the transaction's first Read. A transaction running in Snapshot Isolation is never blocked attempting a read as long as the snapshot data from its *Start-Timestamp* can be maintained. The transaction's writes (updates, inserts, and deletes) will also be reflected in this snapshot, to be read again if the transaction accesses (i.e., reads or updates) the data a second time. Updates by other transactions active after the transaction *Start-Timestamp* are invisible to the transaction.

Snapshot Isolation is a type of multiversion concurrency control. It extends the Multiversion Mixed Method described in [BHG], which allowed snapshot reads by read-only transactions.

When the transaction T1 is ready to commit, it gets a *Commit-Timestamp*, which is larger than any existing *Start-Timestamp* or *Commit-Timestamp*. The transaction successfully commits only if no other transaction T2 with a *Commit-Timestamp* in T1's interval [*Start-Timestamp*, *Commit-Timestamp*] wrote data that T1 also wrote. Otherwise, T1 will abort. This feature, called *First-committer-wins* prevents lost updates (phenomenon P4). When T1 commits, its changes become visible to all transactions whose *Start-Timestamps* are larger than T1's *Commit-Timestamp*.

Theorem

An item-operation history satisfies SI iff its dependency graph is internally consistent and every cycle has two adjacent RW edges.

This exact correspondence is the baseline that predicates must recover.

Review: Downstream Tasks

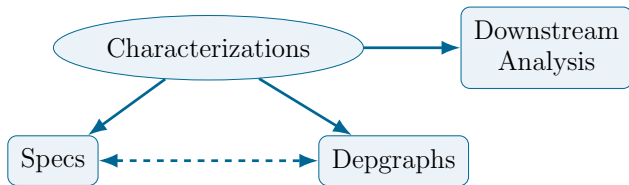
Task	Representative work
Black-box checking	Elle, COBRA, PolySI, VeriStrong
CC protocol verification	execution-based verification
Robustness	robustness against weaker consistency

All depend on a faithful link between observed histories and consistency.

Kingsbury and Alvaro, 2020; Tan et al., 2020; Huang et al., 2023; Cai et al., 2025; Bernardi and Gotsman, 2016

Review (Summary)

What we know about transactional consistency models
without predicates



What we know about transactional consistency models
with predicates

Review: Intuitive or Operational Specs

Eswaran et al., 1976

The Notions of Consistency and Predicate Locks in a Database System

K.P. Eswaran, J.N. Gray,
R.A. Lorie, and I.L. Traiger
IBM Research Laboratory
San Jose, California

SER

serial execution

Berenson et al., 1995

A Critique of ANSI SQL Isolation Levels

Hal Berenson	Microsoft Corp.	haroldb@microsoft.com
Phil Bernstein	Microsoft Corp.	philbe@microsoft.com
Jim Gray	U.C. Berkeley	gray@cs.berkeley.edu
Jim Melton	Sybase Corp.	jim.melton@sybase.com
Elizabeth O'Neil	UMass/Boston	oneil@cs.umb.edu
Patrick O'Neil	UMass/Boston	poneil@cs.umb.edu

4.2 Snapshot Isolation

A transaction executing with *Snapshot Isolation* always reads data from a snapshot of the (committed) data as of the time the transaction started, called its *Start-Timestamp*. This time may be any time before the transaction's first Read. A transaction running in Snapshot Isolation is never blocked attempting a read as long as the snapshot data from its *Start-Timestamp* can be maintained. The transaction's writes (updates, inserts, and deletes) will also be reflected in this snapshot, to be read again if the transaction accesses (i.e., reads or updates) the data a second time. Updates by other transactions active after the transaction *Start-Timestamp* are invisible to the transaction.

Snapshot Isolation is a type of multiversion concurrency control. It extends the Multiversion Mixed Method described in [BHG], which allowed snapshot reads by read-only transactions.

When the transaction T1 is ready to commit, it gets a *Commit-Timestamp*, which is larger than any existing *Start-Timestamp* or *Commit-Timestamp*. The transaction successfully commits only if no other transaction T2 with a *Commit-Timestamp* in T1's interval [*Start-Timestamp*, *Commit-Timestamp*] wrote data that T1 also wrote. Otherwise, T1 will abort. This feature, called *First-committer-wins* prevents lost updates (phenomenon P4). When T1 commits, its changes become visible to all transactions whose *Start-Timestamps* are larger than T1's *Commit-Timestamp*.

SI

snapshot + write-conflict control

Still applicable

Review: Axiomatic Specs

Bouajjani, Enea, and Román-Calvo, 2025

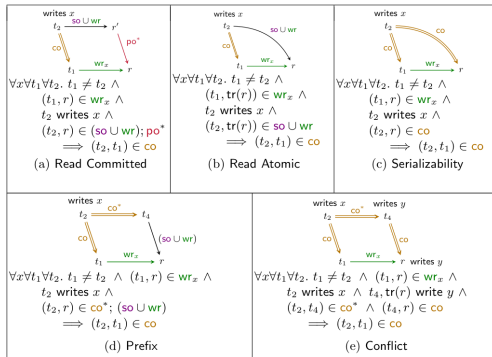
On the Complexity of Checking Mixed Isolation Levels for SQL Transactions

Ahmed Bouajjani¹, Constantin Enea², and Enrique Román-Calvo¹



¹ Université Paris Cité, CNRS, IRIF
{abou, calvo}@irif.fr

² LIX, École Polytechnique, CNRS and
Institut Polytechnique de Paris
cenea@lix.polytechnique.fr



checking-oriented consistency axioms

Review: Dependency Graphs

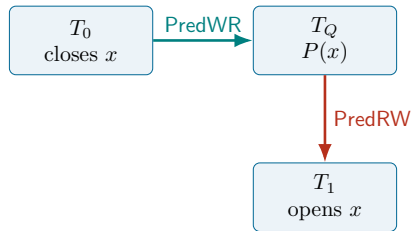
Adya, 1999

**Weak Consistency: A Generalized Theory and Optimistic
Implementations for Distributed Transactions**

by
Atul Adya

Submitted to the Department of Electrical Engineering and Computer Science
in partial fulfillment of the requirements for the degree of

Doctor of Philosophy



A predicate read depends on a witness and
on later result-changing writers.

Review: Dependency Graphs

“Generalized Isolation Level
Definitions” 2000

Generalized Isolation Level Definitions

Atul Adya

Microsoft Research,
1 Microsoft Way,
Redmond, WA 98007
adya@microsoft.com

Barbara Liskov

Laboratory for Computer Science,
Massachusetts Inst. of Technology,
Cambridge, MA 02139
liskov@cs.mit.edu

Patrick O’Neil

Univ. of Massachusetts,
Boston, MA 02125-3393
poneil@cs.umb.edu

generalized isolation

Sun and Zou, 2025

Vbox: Efficient Black-Box Serializability Verification

Anonymous Author(s)

earliest result change

Bouajjani, Enea, and Román-Calvo,
2025

**On the Complexity of Checking Mixed Isolation
Levels for SQL Transactions**

Ahmed Bouajjani¹, Constantin Enea², and Enrique Román-Calvo¹



¹ Université Paris Cité, CNRS, IRIF
{abou, calvo}@irif.fr

² LIX, École Polytechnique, CNRS and
Institut Polytechnique de Paris
cenea@lix.polytechnique.fr

all later writers

Predicate-read dependency definitions differ.

Review: Dependency Graphs

Sun and Zou, 2025

Vbox: Efficient Black-Box Serializability Verification

Anonymous Author(s)

earliest result change

“Generalized Isolation Level
Definitions” 2000

Generalized Isolation Level Definitions

Atul Adya

Microsoft Research,
1 Microsoft Way,
Redmond, WA 98007
adya@microsoft.com

Barbara Liskov

Laboratory for Computer Science,
Massachusetts Inst. of Technology,
Cambridge, MA 02139
liskov@lcs.mit.edu

Patrick O’Neil

Univ. of Massachusetts,
Boston, MA 02125-3393
poneil@cs.umb.edu

generalized isolation

Bouajjani, Enea, and Román-Calvo,
2025

**On the Complexity of Checking Mixed Isolation
Levels for SQL Transactions**

Ahmed Bouajjani¹, Constantin Enea², and Enrique Román-Calvo¹



¹ Université Paris Cité, CNRS, IRIF
{abou, calvo}@irif.fr

² LIX, École Polytechnique, CNRS and
Institut Polytechnique de Paris
cenea@lix.polytechnique.fr

all later writers

$$\text{PredRW}^+ \subseteq \text{PredRW} \subseteq \text{PredRW}^-$$

predicate-anti-dependency definitions differ.

Review: SER Characterization

Predicate extensions use different edges, but lack a single rigorous semantic proof bridge.

3.4 Correctness and Flexibility of the New Specifications

Our conditions for PL-3 provide the well-known notion of conflict-serializability [BHG87, GR93]. That is, $DSG(H)$ is acyclic, and G1a, and G1b are satisfied for a history H iff H is conflict-serializable. We can prove the equivalence of our PL-3 conditions with conflict-serializability using the proof given in [GR93]; our DSGs are similar to their graphs. This equivalence can also be proved along the lines of the proof given in [BHG87] for view-serializability.

Bernstein, Hadzilacos, and Goodman, 1987; Gray and Reuter, 1993; Adya, 1999; “Generalized Isolation Level Definitions” 2000; Fekete et al., 2005; Clark, Donaldson, et al., 2024; Clark, Rigger, et al., 2026; Sun and Zou, 2025

Weak Consistency: A Generalized Theory and Optimistic Implementations for Distributed Transactions

by

Atul Adya

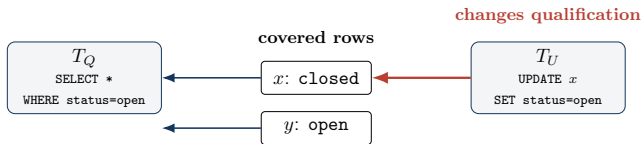
Submitted to the Department of Electrical Engineering and Computer Science
in partial fulfillment of the requirements for the degree of

Doctor of Philosophy

Low level info: Which predicate edges
preserve this exact characterization?

Adya, 1999

The Problem



result: `{y}`

Key observation: $x \notin \text{result}$
still records an observation of x .

Item read: value observed

Predicate read: returned
and excluded items

A phantom changes the
result through absence.

Can one predicate-aware foundation support both specifications and graphs?